Experiment 6

PL/SQL Control Structures, Loops and Exception Handling
Oracle Database 21c XE

1. Source Code

```
-- ------------------------------------------------------------
-- 6a. Nested IF, CASE statement, CASE expression, NULLIF and COALESCE
-- ------------------------------------------------------------

-- 6a-1. Nested IF, CASE, CASE expression, NULLIF and COALESCE
SET SERVEROUTPUT ON
DECLARE
  v_name   VARCHAR2(30) := 'Ravi';
  v_marks  NUMBER := 78;
  v_grade  CHAR(1);
  v_a      NUMBER := 80;
  v_b      NUMBER := 80;
  v_result NUMBER;
BEGIN
  -- Nested IF
  IF v_marks >= 40 THEN
    DBMS_OUTPUT.PUT_LINE(v_name || ' has Passed');
    IF v_marks >= 60 THEN
      DBMS_OUTPUT.PUT_LINE('Result: First Class');
    ELSE
      DBMS_OUTPUT.PUT_LINE('Result: Second Class');
    END IF;
  ELSE
    DBMS_OUTPUT.PUT_LINE(v_name || ' has Failed');
  END IF;

  -- CASE statement
  CASE
    WHEN v_marks >= 90 THEN DBMS_OUTPUT.PUT_LINE('Grade (CASE statement): A');
    WHEN v_marks >= 75 THEN DBMS_OUTPUT.PUT_LINE('Grade (CASE statement): B');
    WHEN v_marks >= 60 THEN DBMS_OUTPUT.PUT_LINE('Grade (CASE statement): C');
    WHEN v_marks >= 40 THEN DBMS_OUTPUT.PUT_LINE('Grade (CASE statement): D');
    ELSE DBMS_OUTPUT.PUT_LINE('Grade (CASE statement): F');
  END CASE;

  -- CASE expression
  v_grade := CASE
               WHEN v_marks >= 90 THEN 'A'
               WHEN v_marks >= 75 THEN 'B'
               WHEN v_marks >= 60 THEN 'C'
               WHEN v_marks >= 40 THEN 'D'
               ELSE 'F'
             END;
  DBMS_OUTPUT.PUT_LINE('Grade (CASE expression): ' || v_grade);

  -- NULLIF
  v_result := NULLIF(v_a, v_b);
  DBMS_OUTPUT.PUT_LINE('NULLIF(80,80) = ' || NVL(TO_CHAR(v_result), 'NULL'));

  -- COALESCE
  DBMS_OUTPUT.PUT_LINE('COALESCE result: ' ||
                       COALESCE(NULL, NULL, 'Oracle', 'SQL'));
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

-- ------------------------------------------------------------
-- 6b. Loops, built-in, user-defined exceptions and RAISE_APPLICATION_ERROR
-- ------------------------------------------------------------

-- 6b-1. Create the student_info table
DROP TABLE student_info;
CREATE TABLE student_info (
  student_id   NUMBER(4) PRIMARY KEY,
  student_name VARCHAR2(30),
  marks        NUMBER(3),
```

```
  age          NUMBER(2)
);
INSERT INTO student_info VALUES (101,'Rahul',82,20);
INSERT INTO student_info VALUES (102,'Sneha',74,19);
INSERT INTO student_info VALUES (103,'Arjun',58,17);
INSERT INTO student_info VALUES (104,'Priya',105,21);
INSERT INTO student_info VALUES (105,'Kiran',45,22);
COMMIT;

-- 6b-2. Run 1: v_id = 101 (all validations pass)
SET SERVEROUTPUT ON
DECLARE
  v_id     NUMBER := 101;          -- change to 999, 104, 103 for the other runs
  v_name   student_info.student_name%TYPE;
  v_marks  student_info.marks%TYPE;
  v_age    student_info.age%TYPE;
  v_count  NUMBER := 1;
  invalid_marks EXCEPTION;
BEGIN
  -- 1. WHILE loop
  DBMS_OUTPUT.PUT_LINE('--- WHILE LOOP ---');
  WHILE v_count <= 5 LOOP
    DBMS_OUTPUT.PUT_LINE(v_count);
    v_count := v_count + 1;
  END LOOP;

  -- 2. Numeric FOR loop
  DBMS_OUTPUT.PUT_LINE('--- FOR LOOP ---');
  FOR i IN 1..5 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
  END LOOP;

  -- 3. Nested loops : multiplication tables 1 to 3
  DBMS_OUTPUT.PUT_LINE('--- MULTIPLICATION TABLES ---');
  FOR i IN 1..3 LOOP
    FOR j IN 1..5 LOOP
      DBMS_OUTPUT.PUT_LINE(i || ' x ' || j || ' = ' || i*j);
    END LOOP;
  END LOOP;

  -- 4. SELECT INTO (may raise NO_DATA_FOUND)
  SELECT student_name, marks, age
  INTO   v_name, v_marks, v_age
  FROM   student_info
  WHERE  student_id = v_id;
  DBMS_OUTPUT.PUT_LINE('Student: ' || v_name || ', Marks: ' ||
                       v_marks || ', Age: ' || v_age);

  -- 5. User-defined exception
  IF v_marks > 100 THEN
    RAISE invalid_marks;
  END IF;

  -- 6. RAISE_APPLICATION_ERROR
  IF v_age < 18 THEN
    RAISE_APPLICATION_ERROR(-20001, 'Age should be 18 or above.');
  END IF;

  DBMS_OUTPUT.PUT_LINE('All validations passed.');
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Student record not found.');
  WHEN invalid_marks THEN
    DBMS_OUTPUT.PUT_LINE('Invalid marks: marks cannot exceed 100.');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

-- 6b-3. Run 2: v_id = 999 (NO_DATA_FOUND)
-- v_id NUMBER := 999;

-- 6b-4. Run 3: v_id = 104 (user-defined exception, marks > 100)
-- v_id NUMBER := 104;

-- 6b-5. Run 4: v_id = 103 (RAISE_APPLICATION_ERROR, age < 18)
-- v_id NUMBER := 103;
```

2. Execution and Output

Output shown is sample output of the programs above (Oracle Database 21c XE, SQL*Plus).

6a. Nested IF, CASE statement, CASE expression, NULLIF and COALESCE

```
[6a-1]
Ravi has Passed
Result: First Class
Grade (CASE statement): B
Grade (CASE expression): B
NULLIF(80,80) = NULL
COALESCE result: Oracle

PL/SQL procedure successfully completed.
```

6b. Loops, built-in, user-defined exceptions and RAISE_APPLICATION_ERROR

```
[6b-1]
Table dropped.

Table created.

1 row created.
1 row created.
1 row created.
1 row created.
1 row created.

Commit complete.

[6b-2]
--- WHILE LOOP ---
1
2
3
4
5
--- FOR LOOP ---
1
2
3
4
5
--- MULTIPLICATION TABLES ---
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
Student: Rahul, Marks: 82, Age: 20
All validations passed.

PL/SQL procedure successfully completed.

[6b-3]
(loops print the same output as Run 1)
Student record not found.

PL/SQL procedure successfully completed.

[6b-4]
(loops print the same output as Run 1)
Student: Priya, Marks: 105, Age: 21
Invalid marks: marks cannot exceed 100.

PL/SQL procedure successfully completed.

[6b-5]
(loops print the same output as Run 1)
Student: Arjun, Marks: 58, Age: 17
Error: ORA-20001: Age should be 18 or above.
```

```
PL/SQL procedure successfully completed.
```